

[TOC]

GitLab On-call Run Books

This project provides a guidance for Infrastructure Reliability Engineers and Managers who are starting an on-call shift or responding to an incident. If you haven't yet, review the Incident Management page in the handbook before reading on.

On-Call

GitLab Reliability Engineers and Managers provide 24x7 on-call coverage to ensure incidents are responded to promptly and resolved as quickly as possible.

Shifts

We use incident.io to manage our on-call schedule and incident alerting. We currently have one schedule for Production Incidents staffed by SREs. We also have

Currently, rotations are weekly and the day's schedule is split 8/8/8 hours with engineers on call as close to daytime hours as their geographical region allows.

Joining the On-Call Rotation

When a new engineer joins the team and is ready to start shadowing for an on-call rotation, they should add themselves to the incident.io schedule on the relevant shadow rotation.

Checklists

- Engineer on Call (EOC)
- Incident Manager on Call (IMOC)
- Communications Manager on Call (CMOC)

To start with the right foot let's define a set of tasks that are nice things to do before you go any further in your week

By performing these tasks we will keep the broken window effect under control, preventing future pain and mess.

Things to keep an eye on

Issues

First check the on-call issues to familiarize yourself with what has been happening lately. Also, keep an eye on the #production and #incident-management channels for discussion around any on-going issues.

Alerts

Start by checking how many alerts are in flight right now

- go to the fleet overview dashboard and check the number of Active Alerts, it should be 0. If it is not 0
 - go to the alerts dashboard and check what is being triggered
 - * gprd prometheus
 - * gprd prometheus-app
 - watch the #production channel for alert notifications; each alert here should point you to the right runbook to fix it.
 - if they don't, you have more work to do.
 - be sure to create an issue, particularly to declare toil so we can work on it and suppress it.

Prometheus targets down

Check how many targets are not scraped at the moment. alerts are in flight right now, to do this:

- go to the fleet overview dashboard and check the number of Targets down. It should be 0. If it is not 0
 - go to the [targets down list] and check what is.
 - * gprd prometheus
 - * gprd prometheus-app
 - try to figure out why there is scraping problems and try to fix it. Note that sometimes there can be temporary scraping problems because of exporter errors.
 - be sure to create an issue, particularly to declare toil so we can work on it and suppress it.

Incidents

First: don't panic.

If you are feeling overwhelmed, escalate to the IMOC. Whoever is in that role can help you get other people to help with whatever is needed. Our goal is to resolve the incident in a timely manner, but sometimes that means slowing down and making sure we get the right people involved. Accuracy is as important or more than speed.

Roles for an incident can be found in the incident management section of the handbook

If you need to declare an incident, follow these instructions located in the handbook.

Communication Tools

If you do end up needing to post and update about an incident, we use Status.io

On status.io, you can Make an incident and Tweet, post to Slack, IRC, Webhooks, and email via checkboxes on creating or updating the incident.

The incident will also have an affected infrastructure section where you can pick components of the GitLab.com application and the underlying services/containers should we have an incident due to a provider.

You can update incidents with the Update Status button on an existing incident, again you can tweet, etc from that update point.

Remember to close out the incident when the issue is resolved. Also, when possible, put the issue and/or google doc in the post mortem link.

Production Incidents

Reporting an incident

Roles

During an incident, we have roles defined in the handbook

General guidelines for production incidents

- Is this an emergency incident?
 - Are we losing data?
 - Is GitLab.com not working or offline?
 - Has the incident affected users for greater than 1 hour?
- Join the **#incident management** channel
- If the *point person* needs someone to do something, give a direct command: *@someone: please run **this** command*
- Be sure to be in sync - if you are going to reboot a service, say so: *I'm bouncing server X*
- If you have conflicting information, **stop and think**, bounce ideas, escalate
- Gather information when the incident is done - logs, samples of graphs, whatever could help figuring out what happened
- use **/security** if you have any security concerns and need to pull in the Security Incident Response team

PostgreSQL

- PostgreSQL
- more postgresql
- PgBouncer

- PostgreSQL High Availability & Failovers
- PostgreSQL switchover
- Read-only Load Balancing
- Add a new secondary replica
- Database backups
- Database backups restore testing
- Rebuild a corrupt index
- Checking PostgreSQL health with postgres-checkup
- Reducing table and index bloat using pg_repack
- Start a read-only psql console
- Maintenance

Frontend Services

- GitLab Pages returns 404
- HAProxy is missing workers
- Worker's root filesystem is running out of space
- GitLab registry is down
- Sidekiq stats no longer showing
- Gemnasium is down
- Blocking a project causing high load

Supporting Services

- Redis
- Sentry is down

Gitaly

- Gitaly error rate is too high
- Gitaly latency is too high
- Sidekiq Queues are out of control
- Workers have huge load because of cat-files
- Test pushing through all the git nodes
- How to gracefully restart gitaly-ruby
- Debugging gitaly with gitaly-debug
- Gitaly token rotation
- Praefect is down
- Praefect error rate is too high

Importers

- Importers runbooks

CI

- Large number of CI pending builds

- The CI runner manager report a high number of errors

Geo

- Geo database replication

ELK

- `mapper_parsing_exception` errors

Non-Critical

- SSL certificate expires
- Troubleshoot git stuck processes

Non-Core Applications

- `version.gitlab.com`

Chef/Knife

- General Troubleshooting
- Error executing action `create` on resource `'directory[/some/path]'`

Certificates

- Certificate runbooks

Learning

Alerting and monitoring

- GitLab monitoring overview
- How to add alerts: Alerts manual
- How to add/update deadman switches
- How to silence alerts
- Alert for SSL certificate expiration
- Working with Grafana
- Working with Prometheus
- Upgrade Prometheus and exporters
- Use `mtail` to capture metrics from logs
- Mixins

CI

- Introduction to Shared Runners
- Understand CI graphs

Access Requests

- Deal with various kinds of access requests

Deploy

- Get the diff between dev versions
- Deploy GitLab.com
- Rollback GitLab.com
- Deploy staging.GitLab.com
- Refresh data on staging.gitlab.com
- Background Migrations
- Migration Skipping

Work with the fleet and the rails app

- Reload Puma with zero downtime
- How to perform zero downtime frontend host reboot
- Gracefully restart sidekiq jobs
- Start a read-only rails console
- Start a rails console in the staging environment
- Start a redis console in the staging environment
- Start a psql console in the staging environment
- Force a failover with postgres
- Force a failover with redis
- Use aptly
- Disable PackageCloud
- Re-index a package in PackageCloud
- Access hosts in GCP

Restore Backups

- Deleted Project Restoration
- PostgreSQL Backups: WAL-E, WAL-G
- Work with GCP Snapshots
- PackageCloud Infrastructure And Recovery

Work with storage

- Understanding GitLab Storage Shards
- How to re-balance GitLab Storage Shards
- Build and Deploy New Storage Servers
- Manage uploads

Mangle front end load balancers

- Isolate a worker by disabling the service in the LBs
- Deny a path in the load balancers

- Purchasing/Renewing SSL Certificates

Work with Chef

- Create users, rotate or remove keys from chef
- Speed up chefspec tests
- Chef Guidelines
- Chef Vault
- Debug failed provisioning

Work with CI Infrastructure

- Runners fleet configuration management
- Investigate Abuse Reports
- Create runners manager for GitLab.com
- Update docker-machine
- CI project namespace check

Work with Infrastructure Providers (VMs)

- Getting Support from GCP

Manually ban an IP or netblock

- Ban a single IP using Redis and Rack Attack
- Ban a netblock on HAProxy

Dealing with Spam

- General procedures for fighting spam in snippets, issues, projects, and comments

ElasticStack (previously Elasticsearch)

Selected elastic documents and resources:

- docs/
 - elastic/
 - * elastic-cloud.md (hosted ES provider docs)
 - * exercises (e.g. cluster performance tuning)
 - * kibana.md
 - * README.md (ES overview)
 - * troubleshooting/
 - README.md (troubleshooting overview)
 - scripts/ (api calls used for admin tasks documented as bash scripts)
 - watchers/

Advanced search integration in Gitlab (indexing Gitlab data)

advanced-search-in-gitlab.md

Zoekt integration in Gitlab (indexing code, BETA)

zoekt-integration-in-gitlab.md

Logging

Selected logging documents and resources:

- docs/
 - logging/
 - * exercises (e.g. searching logs in Kibana)
 - * README.md (logging overview)
 - quick-start
 - what-are-we-logging
 - searching-logs
 - logging-infrastructure-overview
 - * troubleshooting/
 - README.md

Internal DNS

- Managing internal DNS

Debug and monitor

- Tracing the source of an expensive query
- Work with Kibana (logs view)

Secrets

- Working with Google Cloud secrets

Security

- Working with the CloudFlare WAF/CDN
- OSQuery

Other

- Register new domain(s)
- Manage DNS entries
- Setup and Use my Yubikey
- Purge Git data
- Getting Started with Kubernetes and GitLab.com
- Using Chatops bot to run commands across the fleet

Manage Package Signing Keys

- Manage Repository Metadata Signing Keys
- Manage Package Signing Keys

Adding runbooks rules

- Make it quick - add links for checks
- Don't make me think - write clear guidelines, write expectations
- Recommended structure
 - Symptoms - how can I quickly tell that this is what is going on
 - Pre-checks - how can I be 100% sure
 - Resolution - what do I have to do to fix it
 - Post-checks - how can I be 100% sure that it is solved
 - Rollback - optional, how can I undo my fix

Running helper scripts from runbook

Inside of the `bin` directory you can find a list of scripts that can help running repetitive commands or setting up your machine to debug the infrastructure. These scripts can be bash, ruby, python or any other executable.

`glsh` in the single entrypoint to interact with the `bin` directory. For example if you can `glsh hello` it will check if `hello` file exists inside of `bin` directory and execute it. You can also pass multiple arguments, that the script will have access to.

Demo: <https://youtu.be/RsGgxm55YBg>

```
glsh hello arg1 arg2
```

Install

```
git clone git@gitlab.com:gitlab-com/runbooks.git
cd runbooks
sudo make glsh-install
```

Update

```
glsh update
```

Create a new command

1. Create a new file inside of `bin` directory: `touch bin/hello`
2. Populate the file with the contents that you want. The command below updates the file with a simple `echo` command.

```
cat > bin/hello <<EOF
#!/usr/bin/env bash

echo "Hello from glsh"
EOF
```

3. Make it executable: `chmod +x bin/hello`

4. Run it: `glsh hello`

Developing in this repo

Summary

Usually, following a change to the rules, you can test your new additions using:

```
make verify
```

Then, regenerate the rules using:

```
make generate
```

Troubleshooting `make generate`

If you get errors while doing any of these steps try installing any missing dependencies:

```
make jsonnet-bundle
```

You may also run into errors around `jsonnet-tool` not being installed, even though it already is. These seem especially common after upgrading from `asdf` to `mise`. To re-install the tool, run:

```
rm -rf ~/.local/share/mise/downloads/jsonnet-tool
mise plugin add --force jsonnet-tool https://gitlab.com/gitlab-com/gl-infra/asdf-gl-infra.g
```

If the errors persist, read on for more details on how to set up your local environment.

Generating a new runbooks image

To generate a new image you must follow the git commit guidelines below, this will trigger a semantic version bump which will then cause a new pipeline that will build and tag the new image.

:warning: Note that Docker builds only occur when this repo is tagged. When built, we also build the `${CI_DEFAULT_BRANCH}` and `latest` tags. This also means that there's the potential that latest version of our Docker image may not** match the latest code base in the repository.**

Git Commit Guidelines

This project uses Semantic Versioning. We use commit messages to automatically determine the version bumps, so they should adhere to the conventions of Conventional Commits (v1.0.0-beta.2).

TL;DR

- Commit messages starting with **fix:** trigger a patch version bump
- Commit messages starting with **feat:** trigger a minor version bump
- Commit messages starting with **BREAKING CHANGE:** trigger a major version bump.
- If you don't want to publish a new image, do not use the above starting strings.

Automatic versioning

Each push to **master** triggers a **semantic-release** CI job that determines and pushes a new version tag (if any) based on the last version tagged and the new commits pushed. Notice that this means that if a Merge Request contains, for example, several **feat:** commits, only one minor version bump will occur on merge. If your Merge Request includes several commits you may prefer to ignore the prefix on each individual commit and instead add an empty commit summarizing your changes like so:

```
git commit --allow-empty -m '[BREAKING CHANGE|feat|fix]: <changelog summary message>'
```

Tool Versioning

This project has adopted **asdf version-manager** for tool versioning. Using **asdf** is recommended, although not mandatory. Please note that if you chose not to use **asdf**, you'll need to ensure that all the required binaries, and the correct versions, are installed and on your path.

Contributor Onboarding

If you would like to contribute to this project, follow these steps to get your local development environment ready-to-go:

1. Follow the common environment setup steps described in <https://gitlab.com/gitlab-com/gl-infra/common-ci-tasks/-/blob/main/docs/developer-setup.md>.
2. Run the `./scripts/prepare-dev-env.sh` to download and install development dependencies, configure **pre-commit** hooks etc.
3. That's it. You should be ready!

Dependencies and required tooling

Following tools and libraries are required to develop dashboards locally:

- Go programming language
- Ruby programming language
- `go-jsonnet` - Jsonnet implementation written in Go
- `jsonnet-bundler` - package manager for Jsonnet
- `jq` - command line JSON processor

You can install most of them using `asdf` tool.

Manage your dependencies using asdf

Before using `asdf` for the first time, install all the plugins by running:

```
./scripts/install-asdf-plugins.sh
```

Running this command will automatically install the versions of each tool, as specified in the `.tool-versions` file.

```
$ # Confirm everything is working with....
$ asdf current
go-jsonnet      0.16.0   (set by ~/runbooks/.tool-versions)
golang          1.14     (set by ~/runbooks/.tool-versions)
ruby            2.6.5    (set by ~/runbooks/.ruby-version)
```

You don't need to use `asdf`, but in such case you will need install all dependencies manually and track their versions.

Keeping Versions in Sync between GitLab-CI and asdf

`asdf` (and `.tool-versions` generally) is the SSOT for tool versions used in this repository. To keep `.tool-versions` in sync with `.gitlab-ci.yml`, there is a helper script, `./scripts/update-asdf-version-variables.sh`.

Process for updating a tool version

1. Update the version in `.tool-versions`
2. Run `asdf install` to install latest version
3. Run `./scripts/update-asdf-version-variables.sh` to update a refresh of the `.gitlab-ci-asdf-versions.yml` file
4. Commit the changes

Go, Jsonnet

We use `.tool-versions` to record the version of `go-jsonnet` that should be used for local development. The `asdf` version manager is used by some team members to automatically switch versions based on the contents of this file. It should be kept up to date. The top-level `Dockerfile` contains the version of

go-jsonnet we use in CI. This should be kept in sync with `.tool-versions`, and a (non-gating) CI job enforces this.

To install go-jsonnet, you have a few options. We recommend using `asdf` and installing via `./scripts/install-asdf-plugins.sh`.

```
./scripts/install-asdf-plugins.sh
```

Alternatively, you could follow that project's README to install manually. Please ensure that you install the same version as specific in `.tool-versions`.

Or via homebrew:

```
brew install go-jsonnet
```

jsonnet-tool

`jsonnet-tool` is a small home-grown tool for generating configuration from Jsonnet files. The primary reason we use it is because it is much faster than the bash scripts we used to use for the task. Some tasks have gone from 20+ minutes to 2.5 minutes.

We recommend using `asdf` to manage `jsonnet-tool`. The plugin will be installed when

```
# Install jsonnet-tool
./scripts/install-asdf-plugins.sh
# Install the correct version of jsonnet-tool from `.tool-versions`
asdf install
```

Ruby

Ruby is managed through `asdf`. The version of Ruby is configured via the `.tool-versions` file. Note that previously, contributors on this project needed to configure `legacy_version_file = yes` but this setting is no longer required.

Test jsonnet files

There are 2 approaches to write a test for a jsonnet file:

- Use `jsonnetunit`. This method is simple and straight-forward. This approach is perfect for writing unit tests that asserts the output of a particular method. The downside is that it doesn't support jsonnet assertion and inspecting complicated result is not trivial.
- When a jsonnet file becomes more complicated, consists of multiple conditional branches and chains of methods, we should think of writing integration tests for it instead. Jsonnet Unit doesn't serve this purpose very well. Instead, let's use RSpec. Note that we probably don't want to use RSpec for testing small jsonnet functions, the idea would more be for testing

error cases or complicated scenarios where we need to be more expressive about the output we expect

We have two custom matchers for writing integration tests:

```
expect(
  <<~JSONNET
  local grafana = import 'toolinglinks/grafana.libsonnet';

  grafana.grafanaUid("bare-file.jsonnet")
  JSONNET
).to reject_jsonnet(/invalid dashboard path/i)

expect(
  <<~JSONNET
  local grafana = import 'toolinglinks/grafana.libsonnet';

  grafana.grafanaUid("stage-groups/code_review.dashboard.jsonnet")
  JSONNET
).to render_jsonnet('stage-groups-code_review')

# Or a more complicated scenario

expect(
  <<~JSONNET
  local stageGroupDashboards = import 'stage-groups/stage-group-dashboards.libsonnet';

  stageGroupDashboards.dashboard('geo').stageGroupDashboardTrailer()
  JSONNET
).to render_jsonnet { |template|
  expect(template['title']).to eql('Group dashboard: enablement (Geo)')

  expect(template['links']).to match([
    a_hash_including('title' => 'API Detail', 'type' => "dashboards", 'tags' => "type:api"),
    a_hash_including('title' => 'Web Detail', 'type' => "dashboards", 'tags' => "type:web"),
    a_hash_including('title' => 'Git Detail', 'type' => "dashboards", 'tags' => "type:git")
  ])
}

# Or, if you are into matchers

expect(
  <<~JSONNET
  local stageGroupDashboards = import 'stage-groups/stage-group-dashboards.libsonnet';

  stageGroupDashboards.dashboard('geo').stageGroupDashboardTrailer()
  JSONNET
```

```

).to render_jsonnet(
  a_hash_including(
    'title' => eql('Group dashboard: enablement (Geo)'),
    'links' => match([
      a_hash_including('title' => 'API Detail', 'type' => "dashboards", 'tags' => "type:api"),
      a_hash_including('title' => 'Web Detail', 'type' => "dashboards", 'tags' => "type:web"),
      a_hash_including('title' => 'Git Detail', 'type' => "dashboards", 'tags' => "type:git")
    ])
  )
)

```

Location of test files

- JsonnetUnit tests must stay in the same directory and have the same name as the jsonnet file being tested but ending in `_test.jsonnet`. Some examples:
 - `services/stages.libsonnet -> services/stages_test.jsonnet`
 - `libsonnet/toolinglinks/sentry.libsonnet -> libsonnet/toolinglinks/sentry_test.jsonnet`
- RSpec tests replicates the directory structure of the Jsonnet files inside `spec` directory and must end in `_spec.rb` suffixes. Some example:
 - `libsonnet/toolinglinks/grafana.libsonnet -> spec/libsonnet/toolinglinks/grafana_spec.rb`
 - `dashboards/stage-groups/stage-group-dashboards.libsonnet -> spec/dashboards/stage-groups/stage-group-dashboards_spec.rb`

How to run tests?

- Run the full Jsonnet test suite in your local environment with `make test-jsonnet && bundle exec rspec`
- Run a particular Jsonnet unit test file with `scripts/jsonnet_test.sh periodic-queries/periodic-query_test.jsonnet`
- Run a particular Jsonnet integration test file with `bundle exec rspec spec/libsonnet/toolinglinks/grafana_spec.rb`

Note: Verify that you have all the jsonnet dependencies downloaded before attempting to run the tests, you can automatically download the necessary dependencies by running `make jsonnet-bundle`.

Pre-commit hooks

This project supports a set of **pre-commit** hooks which can assist catching CI validation errors before early. While they are not required, they are recommended.

After running the `./scripts/prepare-dev-env.sh` script as described in the Contributor Onboarding section, the **pre-commit** hooks will be automatically installed and ready to go.

When running `git commit`, the hooks will check all staged changes, ensuring that they are valid. The `pre-commit` checks may in some cases automatically fix any problems. If they do this, you'll need to stage the changes and try again.

```
$ git commit
check for case conflicts.....Passed
check that executables have shebangs.....Passed
check json.....(no files to check)Skipped
check for merge conflicts.....Passed
check that scripts with shebangs are executable.....Passed
check for broken symlinks.....(no files to check)Skipped
check yaml.....(no files to check)Skipped
detect private key.....Passed
fix end of files.....Failed
- hook id: end-of-file-fixer
- exit code: 1
- files were modified by this hook
```

Fixing scripts/prepare-dev-env.sh

```
fix utf-8 byte order marker.....Passed
trim trailing whitespace.....Passed
mixed line ending.....Passed
don't commit to branch.....Passed
jsonnetfmt.....(no files to check)Skipped
shellcheck.....Passed
shfmt.....Passed
```

Debug options

Some debug options can be set via environment variables to influence the Jsonnet building process:

- `GL_JSONNET_CACHE_DEBUG`: We cache jsonnet outputs in the `.cache`, setting this variable to `true` prints cache misses on `$stderr`.
- `GL_JSONNET_CACHE_SKIP`: Setting this to `true` disables the jsonnet cache.
- `GL_JSONNET_GNU_PARALLEL`: Setting this to `true` makes `generate-jsonnet-rules` use GNU parallel instead of xargs, and produces a `joblog.txt` file. This can be useful for profiling build times.

Contributing

Please see the contribution guidelines

But always remember

Dont Panic

Figure 1: Dont Panic